

DSP Project 2

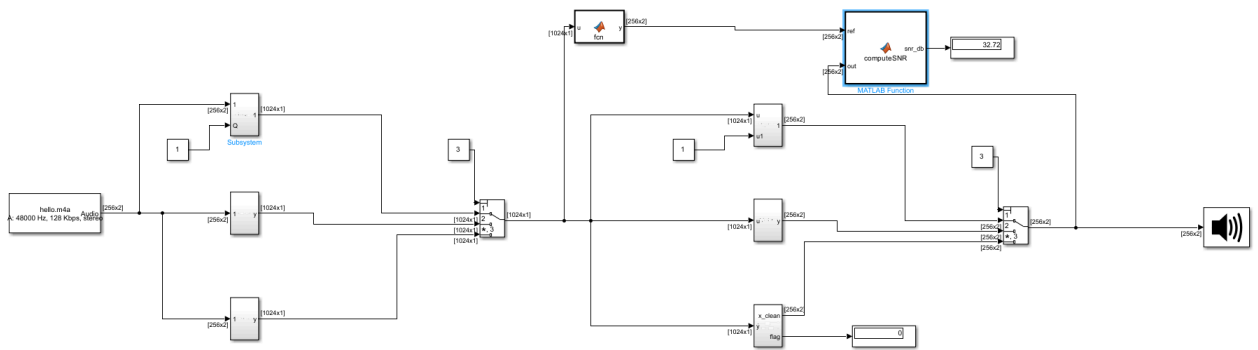
Sachin Kumar Jha 23110286 , Chinmay Kulkarni 23110179, Rupanjan Ghosh 23110282

November 2025

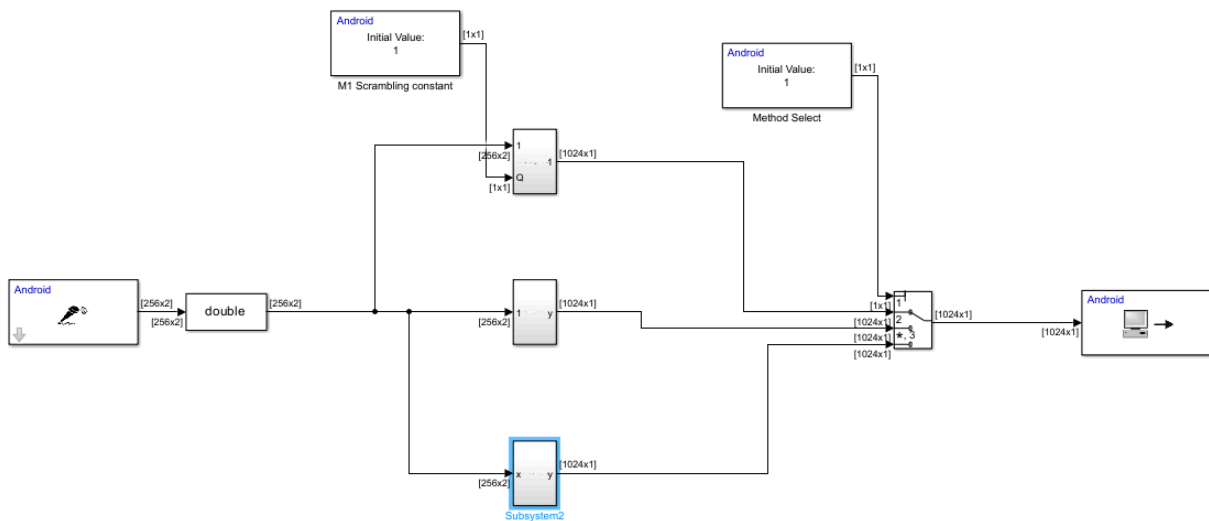
Objective :

Secure and Intelligent Speech Communication over Wi-Fi

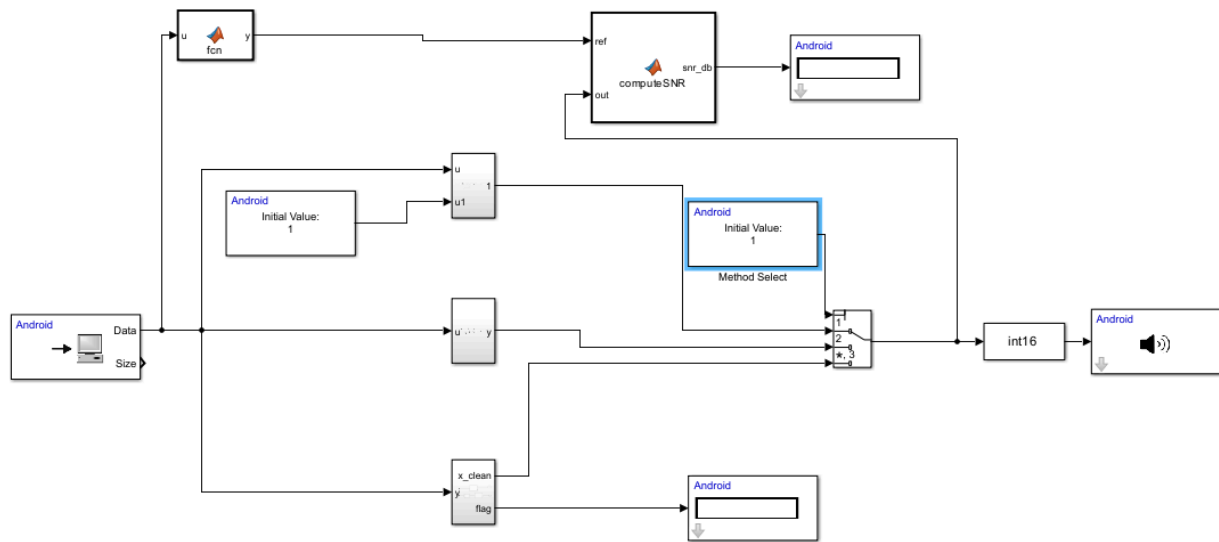
Overall Schematic



Phone A



Phone B



Method 1 : Frequency Scrambling with Key-Based Permutation

This is the exact replica of the previous assignment, except for the inclusion of Gyroscope and Accelerometer data. The details of this method can be found here :

<https://drive.google.com/file/d/1tj2XzpIxrPN6nlrY9pE7bseo3WGCIANJ/view?usp=sharing>

Method 2 : Noise Masking with Adaptive Noise Shaping

- Signal is taken from the microphone and noise is added to it through the AWGN channel.
- A random white noise of 0 mean, is generated from another block.
- The second random noise is adapted to the original signal+noise made.
- FDAF block is used to perform the adaptive filtering.
- Output of the block will be adapted to the only co-related part, which is white noise.
- Output of the FDAF block is hence the adapted noise.
- Input signal is subtracted from FDAF output and the final output is taken.

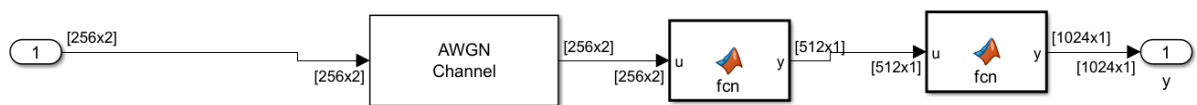
Expectations and Explanations(Noise Masking with Adaptive Noise Shaping) :

- The output is not very effectively filtering out the noise, but the noise which is even coming after the process is not the white noise, which is the original one put in. Hence, it can be said that the original AWGN white noise is removed. There are some distortions, which could be due to the communication channel and also acts as a shaped noise to output. Also the snr display, verifies this claim. Hence, it can be said with reasonable confidence that the objectives were fulfilled.

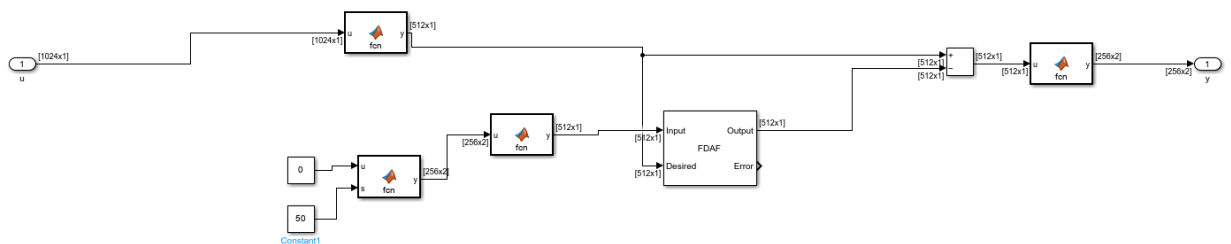
- LMS block was not used since it did not remove white noise from the system, even though its output is much clearer than FDAF block output. The purpose of removing white noise is fulfilled by the FDAF block.

Simulink Block Diagrams

- AWGN Addition and transfer



- Adaptive filtering



Code Generation:

- Random White Noise Generation

```
function y = fcn(u, s)
y = s.*randn(256,2)+u;
```

Method 3 : Watermarking

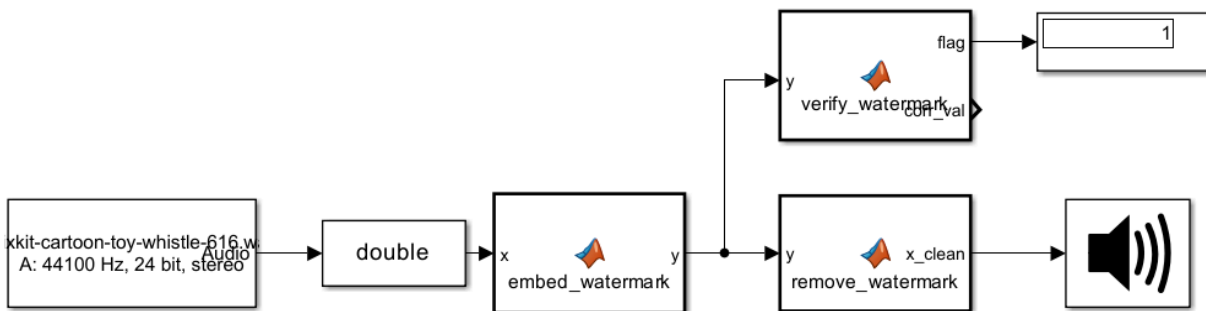
For encryption, we added a 100 Hz sinusoidal signal as watermark in the original audio signal. Its amplitude is kept very low so that it is inaudible while hearing the original signal. For the decryption, we just removed the 100 Hz signal from the encrypted signal and the output signal is same as the original.

For quality metric calculation, we are calculating the SNR. We are first calculating the error by comparing the output signal with the original reference audio. Then we are calculating the noise power and signal power by summing the square of the errors and summing the square of reference points respectively. Then we calculated the SNR using the SNR formula:

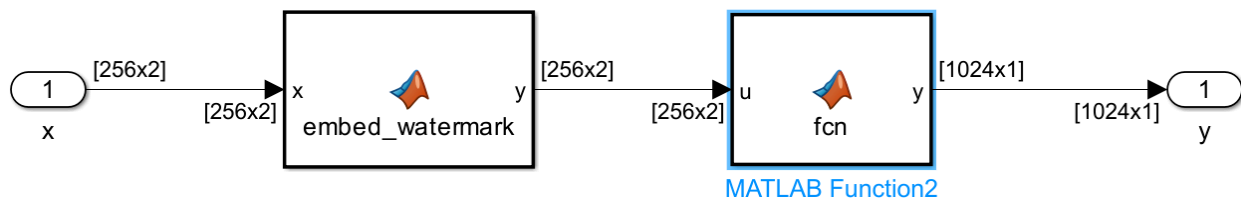
$$SNR = 10 \log_{10} \left(\frac{\text{signal power}}{\text{noise power}} \right)$$

Simulink block diagrams :

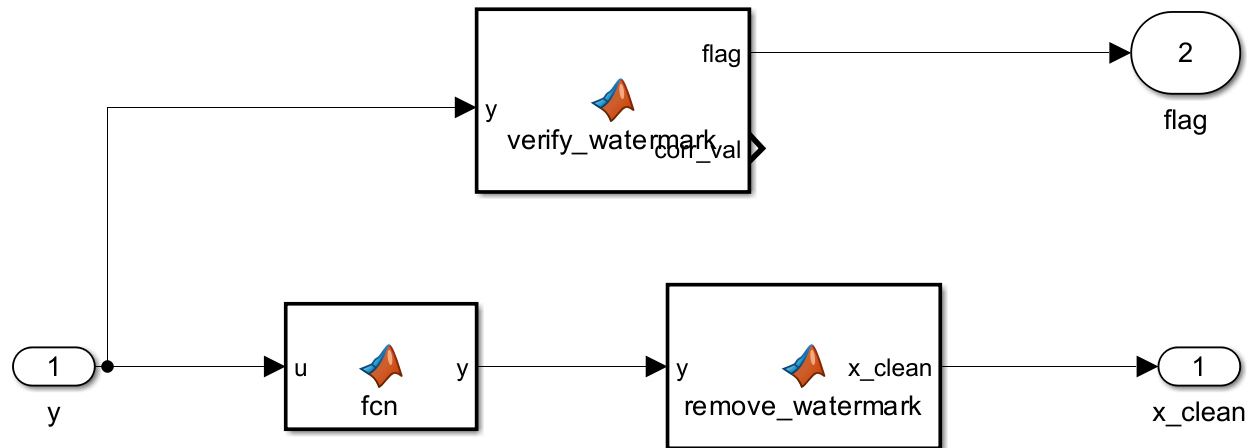
Overall watermark block



Embedding watermark and sending to device B



Removing watermark, verifying the flag and sending to speaker



Embedding code

```

function y = embed_watermark(x)

persistent key amp Fs
if isempty(key)
    key = 12345;
    amp = 0.05;
    Fs = 44100;
end

% --- Watermark Parameters for Audibility ---
watermark_freq = 100; % Low-frequency tone (easy to hear)
watermark_phase = 2*pi * (key / 100000);

[N, C] = size(x);
y = zeros(N, C);
t = (0:N-1)' / Fs;

% --- Generate the Audible Sinusoidal Watermark ---
w = amp * sin(2*pi * watermark_freq * t + watermark_phase);

% --- Embed the Watermark ---
for c = 1:C
    y(:,c) = x(:,c) + w;
    y(:,c) = max(-1, min(1, y(:,c)));
end
end

```

Removing Watermark code

```
function x_clean = remove_watermark(y)

persistent key amp Fs watermark_freq watermark_phase
if isempty(key)
    key = 12345;
    amp = 0.05;
    Fs = 44100;
    watermark_freq = 100;
    watermark_phase = 2*pi * (key / 100000);
end

[N, C] = size(y);
x_clean = zeros(N, C);

% Generate the watermark signal
t = (0:N-1)' / Fs;
w = amp * sin(2*pi * watermark_freq * t + watermark_phase);

% Remove watermark from each channel
for c = 1:C
    x_clean(:,c) = y(:,c) - w;
end

end
```

Verifying Watermark code

```

function [flag, corr_val] = verify_watermark(y)

persistent key amp Fs watermark_freq watermark_phase
if isempty(key)
    key = 12345;
    amp = 0.05;
    Fs = 44100;
    watermark_freq = 100;
    watermark_phase = 2*pi * (key / 100000);
end

[N, C] = size(y);
corr_val = zeros(1, C);

t = (0:N-1)' / Fs;
w_ref = amp * sin(2*pi * watermark_freq * t + watermark_phase);

for c = 1:C
    correlation_matrix = corrcoef(y(:,c), w_ref);
    corr_val(c) = abs(correlation_matrix(1,2));
end
|
threshold = 0.1;
flag = any(corr_val > threshold);
end

```

Observations :

- The output audio signal was not very clear.
- Some noise and echo were there.
- The watermark verification flag was not fixed. It was toggling from 0 to 1 whenever the signal strength crosses the watermark strength.